Object Oriented Programming Using C++

UNIT I — Study Notes

Object Oriented Programming Paradigm & C++ Fundamentals

Contents

1. Object Oriented Programming Paradigm
2. Basic Concepts of OOP (Class, Object, Encapsulation, Abstraction, Inheritance, Polymorphism)
3. Benefits of OOP
4. Object Oriented Languages
5. Applications of OOP
6. The C++ Programming Language
7. Tokens, Keywords, Identifiers and Constants
8. Data Types in C++
9. Type Compatibility
10. Variables
11. Operators in C++
12. Implicit Type Conversions
13. Operator Precedence
14. The Main Function
15. Function Prototyping
16. Call by Reference
17. Return by Reference
18. Inline Functions
19. Function Overloading
20. Friend Functions
21. Default Parameter Value
22. Quick Comparison: C vs C++
23. Solved Conceptual Examples
24. Important Exam Questions — Unit I

Quick Revision Summary & Glossary

1. Object Oriented Programming Paradigm

A programming paradigm is a style or approach used to structure and write code. Early languages such as C used the procedural paradigm, where a program is divided into functions that operate on shared, global data. As programs grew larger, this design made code hard to maintain and insecure because any function could accidentally change any data.

Object Oriented Programming (OOP) solves this by organising a program around objects self-contained units that bundle data (attributes) and the functions that operate on that data (methods). Instead of asking 'what should this function do to the data', OOP asks 'what objects exist in this problem, and how do they interact'.

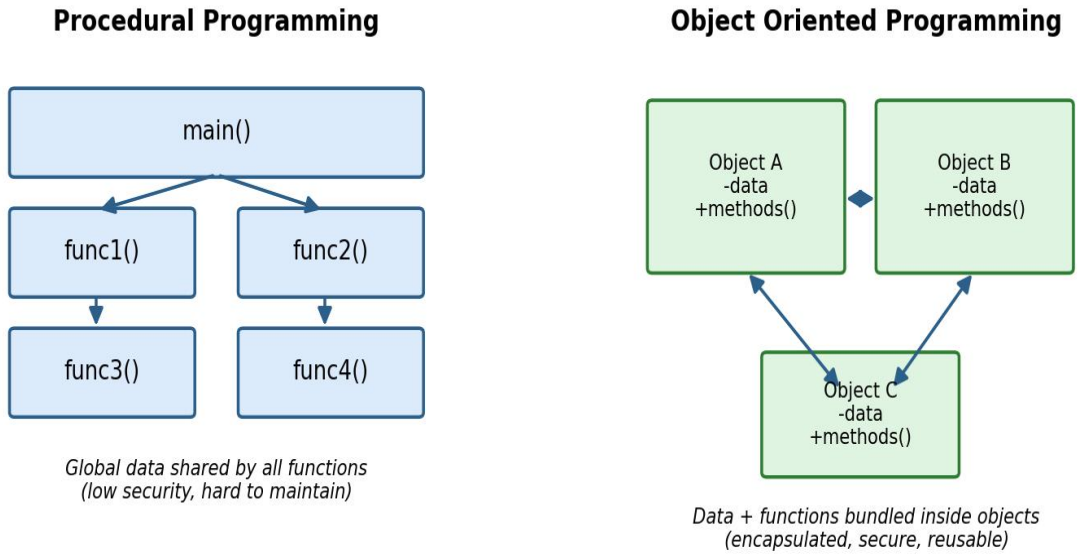


Fig 1.1 — Procedural programming (functions act on shared global data) vs Object Oriented Programming (data and functions are bundled inside independent objects).

Example (real-world analogy)

A 'Car' in real life has attributes (colour, speed, fuel level) and behaviours (start(), accelerate(), brake()). In OOP we model the car exactly this way, as a Car object, instead of writing separate loose functions like startCar(), accelerateCar() that operate on a global car-speed variable.

2. Basic Concepts of Object Oriented Programming

OOP rests on a small set of core ideas. Together these are often called the four pillars of OOP, along with a few supporting concepts.

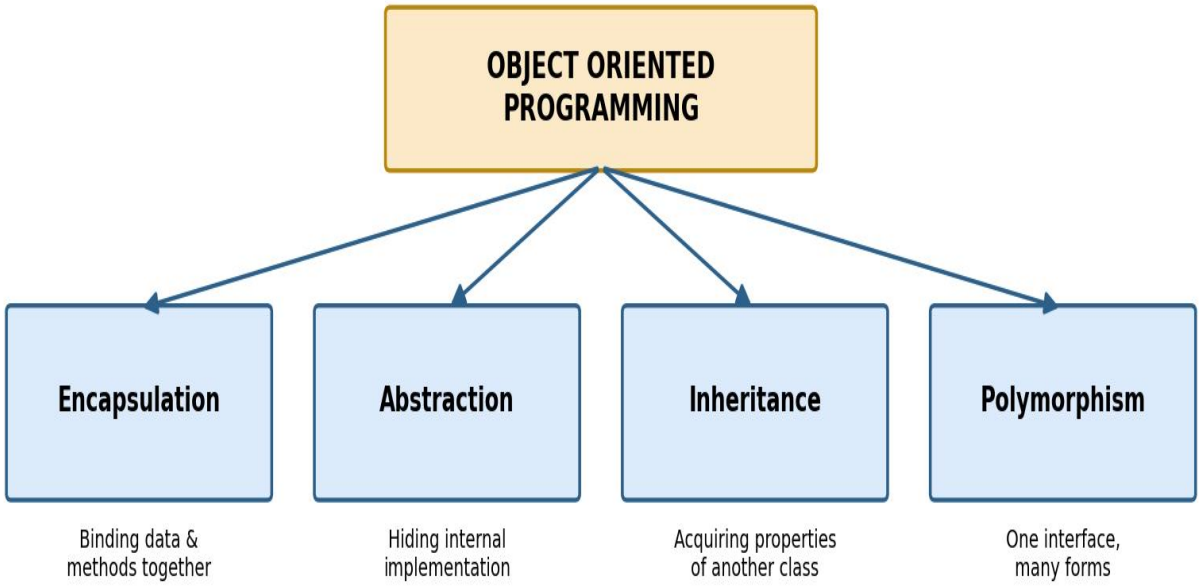


Fig 2.1 — The four pillars of OOP.

2.1 Class

A class is a user-defined blueprint or template that defines the data members (variables) and member functions (methods) common to all objects of that type. No memory is allocated for a class until an object is created from it.

```
class Student {
   private:
      int rollNo;
      string name;
   public:
      void setData(int r, string n) {
         rollNo = r; name = n;
      }
      void display() {
         cout << rollNo << " " << name;
      }
};
```

2.2 Object

An object is a concrete instance of a class. When an object is declared, memory is allocated for its data members, and it can invoke the class's member functions.

```
int main() {
   Student s1;         // object created
   s1.setData(1, "Aman");
   s1.display();       // output: 1 Aman
   return 0;
}
```

Output: 1 Aman

2.3 Encapsulation

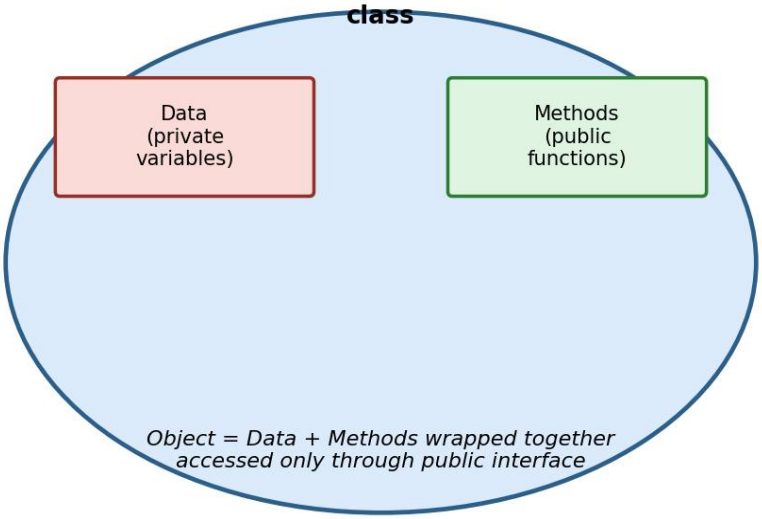


Fig 2.2 — Encapsulation: data and methods enclosed together; data reached only via public methods.

Encapsulation is the wrapping of data and the functions that act on it into a single unit (class), and restricting direct access to the internal data using access specifiers (private, protected). Access is only permitted through public 'getter/setter' style methods — this is also called data hiding.

```
class Account {
   private:
      double balance;      // hidden from outside
   public:
      void deposit(double amt) { balance += amt; }
      double getBalance() { return balance; }
};
// balance cannot be touched directly: obj.balance = 5000; -> ERROR
```

2.4 Abstraction

Abstraction means exposing only the essential features of an object while hiding the background implementation details. A driver uses a steering wheel and pedals (the interface) without needing to know how the engine or braking system works internally (the implementation). In C++, abstraction is achieved using classes, access specifiers, and abstract classes (classes containing at least one pure virtual function).

```
class Shape {                // abstraction
   public:
      virtual double area() = 0;   // pure virtual -> abstract class
};
class Circle : public Shape {
   double r;
   public:
      Circle(double x) { r = x; }
      double area() { return 3.14 * r * r; }  // implementation hidden from user
};
```

2.5 Inheritance

Inheritance allows a new class (derived/child class) to acquire the properties and behaviours of an existing class (base/parent class), promoting code reuse. Full detail and code examples are given in Unit III of the syllabus, but the basic idea is introduced here.

```
class Animal {               // base class
   public: void eat() { cout << "eating..."; }
};
class Dog : public Animal {    // derived class
   public: void bark() { cout << "barking..."; }
};
// Dog object can use both eat() and bark()
```

2.6 Polymorphism

Polymorphism ('many forms') means the same function name or operator behaves differently depending on the context. C++ supports compile-time polymorphism (function overloading, operator overloading) and run-time polymorphism (virtual functions, discussed under Unit III).

```
int add(int a, int b)      { return a + b; }
double add(double a, double b) { return a + b; }
// same name 'add' behaves differently for int and double arguments
```

2.7 Message Passing & Dynamic Binding

- Message Passing: objects communicate by calling each other's member functions (sending a 'message'), e.g. obj1.display() sends a message to obj1 to execute display().
- Dynamic Binding: the function call to be executed is resolved at run time rather than compile time (used with virtual functions).

3. Benefits of Object Oriented Programming

- Modularity: a program is broken into independent objects, making it easier to design, understand and debug.
- Reusability: classes can be reused across programs through inheritance, saving development time.
- Data security: encapsulation and access specifiers protect data from accidental or unauthorised access.
- Easy maintenance: changes to one class rarely affect unrelated classes, so large systems are easier to update.
- Problem-solving that mirrors the real world: objects map naturally onto real-world entities, making designs intuitive.
- Extensibility: new features can be added by creating new classes or extending existing ones through inheritance.
- Polymorphism reduces code complexity by letting one interface serve many underlying implementations.

4. Object Oriented Languages

Languages are generally classified by how fully they support OOP concepts:

Category	Description	Examples
Pure OOP languages	Everything is treated as an object; no support for procedural style.	Smalltalk, Ruby
Hybrid OOP languages	Support both procedural and object oriented styles.	C++, Java*, Python
Object-based languages	Support objects/encapsulation but lack full inheritance/polymorphism.	JavaScript (classic), VB (early)

**Java is sometimes classed as 'pure OOP' at the language-design level even though primitive types exist. C++ is the classic example of a hybrid language — it supports plain procedural C-style code as well as full OOP.*

5. Applications of Object Oriented Programming

- Graphical User Interfaces (GUIs) — windows, buttons, menus are naturally modelled as objects.
- Object-oriented databases and simulation systems (traffic simulation, weather modelling).
- Real-time and embedded systems, where components map to physical devices/objects.
- Computer-Aided Design/Manufacturing (CAD/CAM) systems.
- Game development — characters, weapons, levels are all objects with properties and behaviours.
- Client-server / distributed and web applications, including object-oriented analysis and design (OOAD) of large software systems.
- Artificial Intelligence and expert systems, where knowledge is represented using objects and their relationships.

6. The C++ Programming Language

C++ was developed by Bjarne Stroustrup at Bell Labs (initially called 'C with Classes', released as C++ in 1985) as a superset of the C language, adding object oriented features while retaining C's speed and low-level control. A C++ program typically has this structure:

```
#include <iostream>     // preprocessor directive
using namespace std;

int main() {            // program execution begins here
   cout << "Hello, OOP World!";
   return 0;
}
```

Output: Hello, OOP World!

7. Tokens, Keywords, Identifiers and Constants

7.1 Tokens

A token is the smallest individual unit of a C++ program that the compiler recognises. C++ has five categories of tokens:

- Keywords — e.g. int, class, if, while
- Identifiers — names given to variables, functions, classes
- Constants/Literals — fixed values, e.g. 10, 'A', 3.14, "text"
- Punctuators/Special symbols — e.g. { } ( ) ; ,
- Operators — e.g. +, -, *, ==, &&

7.2 Keywords

Keywords are reserved words that have a predefined meaning in C++ and cannot be used as identifiers. Examples: int, float, double, char, class, public, private, protected, if, else, for, while, return, new, delete, virtual, friend, this, try, catch. C++ has 95+ reserved keywords.

7.3 Identifiers

Identifiers are user-defined names for variables, functions, classes, objects, etc. Rules: must start with a letter or underscore, may contain letters/digits/underscores, cannot be a keyword, and are case-sensitive.

```
int marks;       // valid identifier
int _total;      // valid (starts with underscore)
int 2ndYear;     // INVALID - cannot start with a digit
int class;       // INVALID - 'class' is a keyword
```

7.4 Constants (Literals)

Constants are fixed values that do not change during program execution.

Type	Example
Integer constant	10, -25, 0x1A (hex), 017 (octal)
Floating-point constant	3.14, 2.5e3
Character constant	'A', '9', '\n'
String constant	"Hello"
Boolean constant	true, false

```
const float PI = 3.14159;          // symbolic constant using 'const'
#define MAX 100                    // constant using preprocessor
```

8. Data Types in C++

A data type specifies the kind of value a variable can hold and how much memory it occupies.

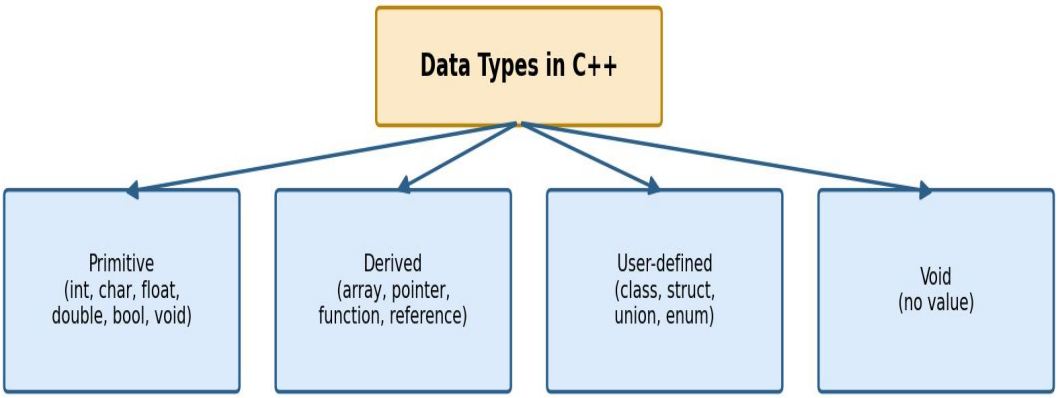


Fig 8.1 — Classification of data types in C++.

Data type	Typical size	Range / use
char	1 byte	single character, -128 to 127
int	4 bytes	whole numbers, ~ -2.1 billion to 2.1 billion
float	4 bytes	single precision decimal, ~6-7 digits
double	8 bytes	double precision decimal, ~15 digits
bool	1 byte	true or false
void	—	represents 'no value' (e.g. void functions)

```
int age = 20;
float price = 99.5f;
double pi = 3.14159265;
char grade = 'A';
bool isPass = true;
```

9. Type Compatibility

Type compatibility refers to whether a value of one data type can be assigned to, or used in an expression with, another data type. C++ is a strongly (but not strictly) typed language:

- Compatible types: values automatically convert without loss, e.g. int to double (widening).
- Incompatible/ risky types: converting double to int (narrowing) silently truncates the decimal part, which can cause loss of data.

```
int a = 10;
double b = a;      // OK - int is 'compatible' with double (widening)
int c = 9.8;       // allowed but c becomes 9 (narrowing, data lost)
```

Output: b = 10 c = 9

10. Variables

A variable is a named memory location used to store a value that may change during program execution. Every variable must be declared with a data type before use.

```
int score;         // declaration
score = 95;        // initialization (assignment)
int total = 0;     // declaration + initialization together
```

Types of variables (by scope/storage)

- Local variables — declared inside a function/block; exist only during that block's execution.
- Global variables — declared outside all functions; accessible throughout the program.
- Static variables — retain their value between function calls (declared with the 'static' keyword).
- Instance variables — data members belonging to each object of a class.

11. Operators in C++

An operator is a symbol that tells the compiler to perform a specific mathematical, relational or logical operation.

Category	Operators	Example
Arithmetic	+ - * / %	a + b, a % b
Relational	== != > < >= <=	a == b
Logical	&& || !	a > 0 && b > 0
Assignment	= += -= *= /=	a += 5
Increment/Decrement	++ --	a++, --b
Bitwise	& | ^ ~ << >>	a & b
Conditional (ternary)	? :	(a>b) ? a : b
Scope resolution	::	Class::member

```
int a = 10, b = 3;
cout << a + b << endl;   // 13
cout << a % b << endl;   // 1
cout << (a > b ? a : b); // 10 (ternary operator)
```

Output: 13 1 10

12. Implicit Type Conversions

Implicit (automatic) type conversion happens when the compiler converts one data type to another on its own, without the programmer writing an explicit cast — usually when a smaller type is mixed with a larger type in an expression, or when a value is assigned to a variable of a different type.

```
int i = 10;
double d = 3.5;
double result = i + d;   // i is implicitly converted to double before addition
cout << result;          // 13.5

char ch = 'A';
int code = ch;           // implicit char -> int conversion (ASCII value)
cout << code;            // 65
```

Output: 13.5 65

Implicit conversion follows a general rule of 'promoting' the smaller/lower-ranked type to the larger/higher-ranked type (bool -> char -> int -> float -> double) to avoid loss of information wherever possible; converting the other way (e.g. double to int) can lose precision and is usually done with an explicit cast instead: int x = (int) 9.7;

13. Operator Precedence

When an expression contains multiple operators, precedence rules decide which operator is evaluated first, and associativity decides the order (left-to-right or right-to-left) among operators of equal precedence.

Precedence (high to low)	Operators	Associativity
1	:: (scope), () [] . ->	Left to right
2	! ~ ++ -- (unary +/-) (type) *ptr &addr	Right to left
3	* / %	Left to right
4	+ -	Left to right
5	<< >>	Left to right
6	< <= > >=	Left to right
7	== !=	Left to right
8	&&	Left to right
9	||	Left to right
10	?: (ternary)	Right to left
11	= += -= *= /=	Right to left

```
int result = 10 + 2 * 3;     // * has higher precedence than +
cout << result;              // 16, NOT 36
```

Output: 16

14. The Main Function

Every executable C++ program must have exactly one main() function — this is the entry point where program execution starts and ends. The operating system calls main() to run the program.

```
int main() {
    cout << "Program starts here";
    return 0;      // 0 signals successful termination to the OS
}

// main() can also accept command-line arguments:
int main(int argc, char* argv[]) {
    cout << "Number of arguments: " << argc;
    return 0;
}
```

15. Function Prototyping

A function prototype (declaration) tells the compiler a function's name, return type and parameter types before the function is actually defined or called, so the compiler can check calls for correctness even if the function body appears later in the file.

```
#include <iostream>
using namespace std;

int add(int, int);      // function prototype

int main() {
    cout << add(4, 5);  // compiler already knows add()'s signature
    return 0;
}

int add(int a, int b) { // function definition (after main)
    return a + b;
}
```

Output: 9

16. Call by Reference

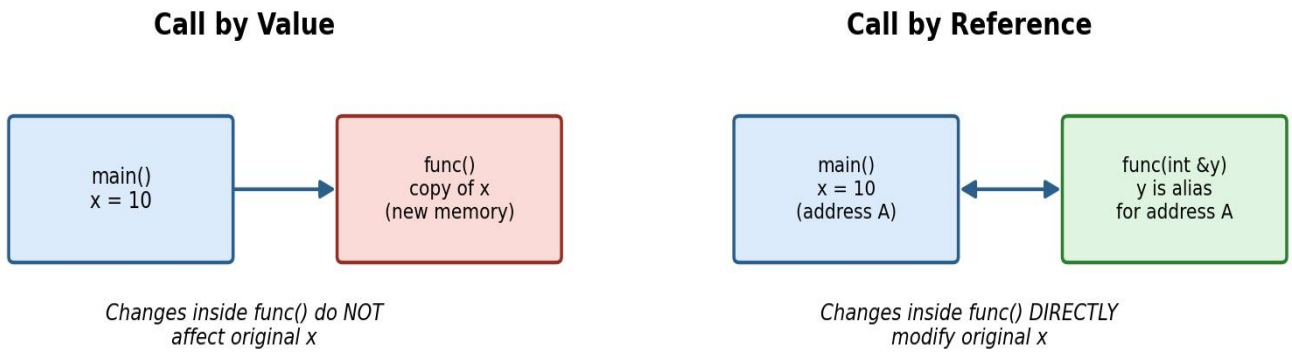


Fig 16.1 — Call by value copies data; call by reference lets the function operate on the caller's original variable.

In call by reference, the function parameter becomes an alias (reference, declared with &) for the caller's original argument, instead of receiving a copy. Any change made to the parameter inside the function directly changes the original variable.

```
void increment(int &x) {   // x is a reference to the caller's variable
   x = x + 1;
}
int main() {
   int num = 5;
   increment(num);
   cout << num;          // 6 (original variable changed!)
   return 0;
}
```

Output: 6

Compare with call by value, where the same increment(int x) with no & would leave num unchanged (still 5), because x would just be a local copy.

17. Return by Reference

A function can also return a reference to a variable instead of returning a copy of its value. This allows the function call itself to be used as an lvalue (i.e. it can appear on the left side of an assignment), and avoids the overhead of copying large objects.

```
int arr[5] = {10, 20, 30, 40, 50};

int& getElement(int index) {   // returns a reference, not a copy
    return arr[index];
}

int main() {
    getElement(2) = 99;        // modifies arr[2] directly!
    cout << arr[2];            // 99
    return 0;
}
```

Output: 99

Caution: never return a reference to a local variable of the function — that variable is destroyed once the function ends, leaving a 'dangling reference'.

18. Inline Functions

Normally, calling a function involves overhead — jumping to the function's memory location, pushing arguments on the stack, then jumping back. For very small, frequently-called functions, this overhead can be significant. Declaring a function inline is a request to the compiler to substitute the function's actual code directly at the point of each call, avoiding the call overhead — similar in spirit to a C macro, but type-safe.

```
inline int square(int x) {
    return x * x;
}
int main() {
    cout << square(5);   // compiler may replace this with: cout << 5*5;
    return 0;
}
```

Output: 25

- inline is only a request/suggestion — the compiler may ignore it for large or recursive functions.
- Best suited for small functions used repeatedly (e.g. simple getters, small math utilities).
- Trade-off: faster execution (no call overhead) vs larger executable size (code is duplicated at every call site).

19. Function Overloading

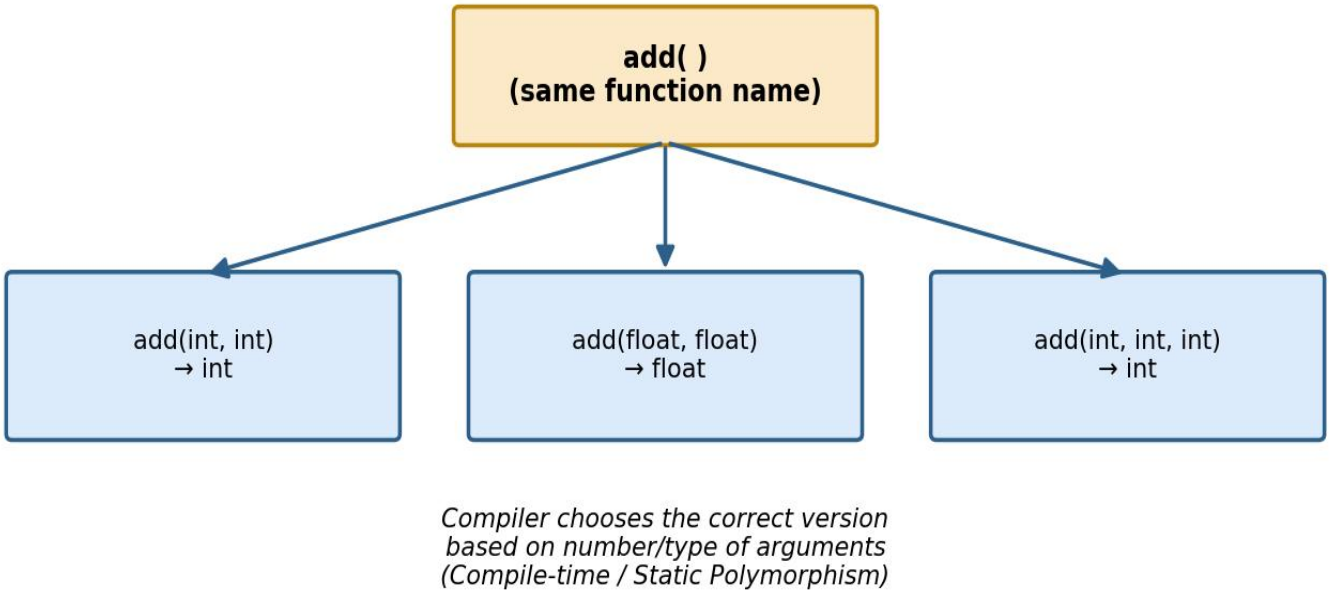


Fig 19.1 — Multiple functions share the same name but differ in their parameter list.

Function overloading allows two or more functions in the same scope to share the same name, as long as their parameter lists differ in number of parameters, type of parameters, or order of parameters. The compiler selects the correct version to call at compile time based on the arguments passed — this is compile-time (static) polymorphism. Note: the return type alone cannot distinguish overloaded functions.

```
int add(int a, int b) {
    return a + b;
}
double add(double a, double b) {
    return a + b;
}
int add(int a, int b, int c) {
    return a + b + c;
}
int main() {
    cout << add(2, 3) << endl;         // calls int version -> 5
    cout << add(2.5, 3.5) << endl;     // calls double version -> 6.0
    cout << add(1, 2, 3) << endl;      // calls 3-parameter version -> 6
    return 0;
}
```

Output: 5 6 6

20. Friend Functions

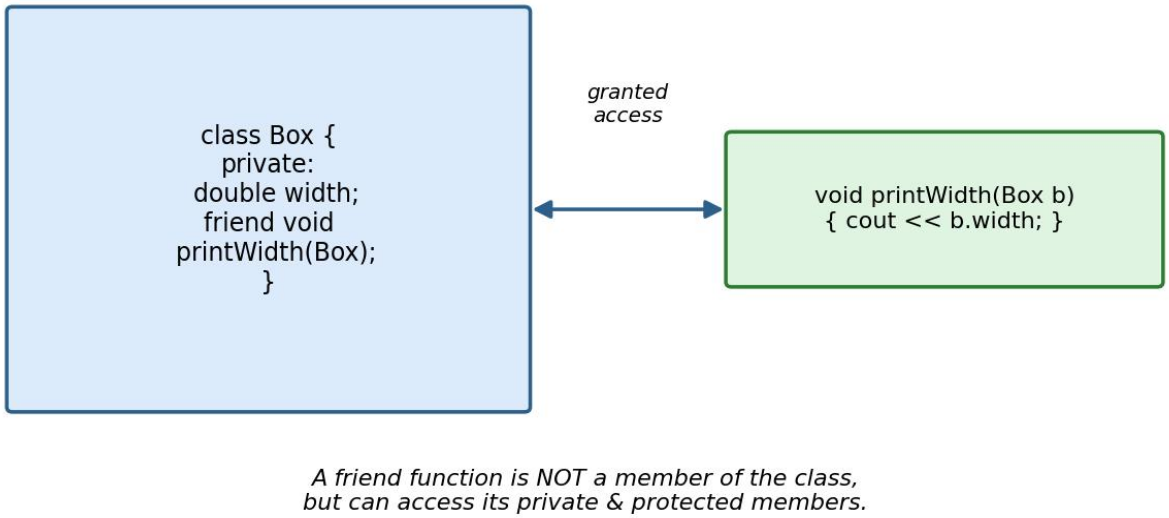


Fig 20.1 — A friend function is declared inside the class but defined outside, and can access private members.

Private and protected members of a class normally cannot be accessed from outside the class. A friend function is a non-member function that is explicitly granted permission (using the 'friend' keyword inside the class) to access the class's private and protected members. A friend function is NOT a member of the class, so it is not called using the dot operator and does not have a 'this' pointer.

```
class Box {
   private:
      double width;
   public:
      Box(double w) { width = w; }
      friend void printWidth(Box b);   // friend declaration
};

void printWidth(Box b) {               // NOT a member function
   cout << "Width: " << b.width;       // yet can access private 'width'
}

int main() {
   Box b(12.5);
   printWidth(b);
   return 0;
}
```

Output: Width: 12.5

- A friend function can be a global function, or a member function of another class.
- A whole class can be made a friend class, giving all of its member functions access.
- Friendship is not mutual and not inherited: if A is a friend of B, B is not automatically a friend of A, and A's derived classes do not inherit the friendship.

21. Default Parameter Value

C++ allows a function to specify default values for its parameters. If the caller omits an argument, the default value is used automatically. Default values must be specified from right to left — once one parameter has a default, all parameters after it must also have defaults.

```
void greet(string name, string msg = "Welcome!") {
   cout << name << ": " << msg << endl;
}
int main() {
   greet("Aman");                 // uses default msg
   greet("Riya", "Good morning"); // overrides default
   return 0;
}
```

Output: Aman: Welcome! Riya: Good morning

Default parameters are especially useful together with function overloading, since they let a single function serve the role of several overloaded versions with fewer arguments.

22. Quick Comparison: C vs C++

Since C++ is built on top of C, it is useful to be clear about what C++ adds. This comparison is a common short-answer exam question.

Feature	C	C++
Paradigm	Procedural only	Procedural + Object Oriented
Data security	No access specifiers; data is global/shared	private/protected/public + encapsulation
Function overloading	Not supported	Supported
Reference variables	Not supported (only pointers)	Supported (&)
Standard I/O	printf() / scanf()	cin / cout (also supports printf/scanf)
Inheritance	Not supported	Supported
Exception handling	Not built-in	try / catch / throw
Standard Template Library	Not available	Available (vectors, maps, algorithms, etc.)

23. Solved Conceptual Examples

Example 1 — Predict the output (Operator precedence)

```
int a = 5, b = 10, c = 2;
int result = a + b * c - (a + b) / c;
cout << result;
```

*Solution: b*c = 20, (a+b) = 15, 15/c = 7 (integer division). So result = 5 + 20 - 7 = 18.*

Output: 18

Example 2 — Call by value vs call by reference

```
void byValue(int x)  { x = x * 2; }
void byRef(int &x)   { x = x * 2; }
int main() {
   int a = 5, b = 5;
   byValue(a);
   byRef(b);
   cout << a << " " << b;
}
```

Solution: byValue() only modifies its local copy, so 'a' stays 5. byRef() modifies the caller's variable directly through the reference, so 'b' becomes 10.

Output: 5 10

Example 3 — Which overloaded function is called?

```
void show(int x)    { cout << "int version\n"; }
void show(double x) { cout << "double version\n"; }
int main() {
   show(10);      // ?
   show(10.5);    // ?
   show('A');     // ?  ('A' has no exact char overload)
}
```

Solution: show(10) matches the int version exactly. show(10.5) matches double exactly. show('A') has no char overload, so the compiler implicitly promotes char to int and calls the int version.

Output: int version double version int version

Example 4 — sizeof() of primitive data types

```
cout << sizeof(char) << " " << sizeof(int) << " "
    << sizeof(float) << " " << sizeof(double);
```

Solution: on a typical 32/64-bit system, char = 1 byte, int = 4 bytes, float = 4 bytes, double = 8 bytes (sizes are compiler/platform dependent, but these are the standard values expected in exams).

Output: 1 4 4 8

Example 5 — Friend function access

```
class Rectangle {
   int length, breadth;
 public:
   Rectangle(int l, int b) { length = l; breadth = b; }
   friend int area(Rectangle r);
};
int area(Rectangle r) { return r.length * r.breadth; }
int main() {
   Rectangle r(4, 5);
   cout << area(r);
}
```

Solution: area() is declared a friend inside Rectangle, so even though it is not a member function it can directly read the private members length and breadth.

Output: 20

24. Important Exam Questions — Unit I

Based on the paper pattern given in the syllabus (Q1 = compulsory short-answer covering the whole syllabus; remaining questions = 2 per unit, attempt 1, up to 5 sub-parts):

A. Short answer / objective type (for compulsory Question 1)

- Define Object Oriented Programming. How is it different from procedural programming?
- List any four benefits of OOP.
- What is the difference between a class and an object?
- Define encapsulation with one real-life example.
- What is an identifier? State any two rules for naming identifiers in C++.
- Differentiate between keywords and identifiers.
- What is the purpose of the main() function in C++?
- What is a function prototype? Why is it needed?
- Differentiate between call by value and call by reference.
- What is an inline function? When should it be used?
- What is function overloading? Give one example.
- What is a friend function? Can it access private members of a class?
- What is a default parameter? Give an example.
- What is implicit type conversion? Give an example.
- State the precedence order between *, +, and = operators.

B. Long answer type (unit questions, ~15 marks each)

- Explain the basic concepts of Object Oriented Programming (class, object, encapsulation, abstraction, inheritance, polymorphism) with suitable examples for each.
- What are the benefits and applications of OOP? Explain with real-world examples.
- Explain tokens in C++. Describe keywords, identifiers and constants with examples of each.
- Explain the different data types available in C++ with their size and range.
- What is type compatibility? Explain implicit type conversion with a program.
- Explain call by reference and return by reference with suitable C++ programs. What are the risks of returning a reference?
- What is function overloading? Write a C++ program to overload a function to find the area of a square, rectangle and circle.
- Explain the concept of friend functions with an example program. What are its advantages and limitations?
- Explain inline functions and default parameter values with suitable examples. Discuss their advantages and disadvantages.

Quick Revision Summary — Unit I

- OOP organises programs as interacting objects (data + methods) instead of loose functions on shared data.
- Four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.
- Benefits: modularity, reusability, security, easier maintenance, extensibility.
- C++ is a hybrid language — supports both procedural (C-style) and object-oriented programming.
- Tokens = keywords + identifiers + constants + punctuators + operators.
- Data types: primitive (int, char, float...), derived (array, pointer...), user-defined (class, struct...).
- Implicit conversion happens automatically; explicit conversion (casting) is done by the programmer.
- Operator precedence decides evaluation order when multiple operators appear in one expression.
- main() is the mandatory entry point of every C++ program.
- Call by reference (&) lets a function modify the caller's original variable; return by reference lets a function call be used as an lvalue.
- inline requests the compiler to expand a function's code at the call site to remove call overhead.
- Function overloading = same function name, different parameter list, resolved at compile time.
- friend functions/classes get special access to a class's private members without being members themselves.
- Default parameter values let arguments be optional in a function call.

Glossary of Key Terms

Term	Meaning
Class	Blueprint/template defining data members and member functions.
Object	Runtime instance of a class, occupying its own memory.
Encapsulation	Bundling data and methods together and hiding data behind access specifiers.
Abstraction	Showing only essential features and hiding implementation detail.
Inheritance	Deriving a new class from an existing class to reuse its members.
Polymorphism	Same name/interface behaving differently in different contexts.
Token	Smallest meaningful unit recognised by the compiler.
Keyword	Reserved word with a fixed, predefined meaning.
Identifier	Programmer-chosen name for a variable, function, class, etc.
Constant / Literal	A fixed value that does not change at run time.
Variable	Named memory location holding a value that can change.
Operator precedence	Rule deciding which operator is evaluated first in an expression.
Type conversion	Changing a value from one data type to another (implicit or explicit).
Call by reference	Passing the address/alias of an argument so the function can modify the original.
Function prototype	Forward declaration of a function's name, return type and parameters.
Inline function	Function whose code the compiler may insert directly at the call site.
Function overloading	Multiple functions with the same name but different parameter lists.
Friend function	Non-member function granted access to a class's private/protected members.
Default parameter	A parameter that takes a preset value if the caller omits it.